

Atouts du relationnel

- Très bon support théorique
- Simplicité des concepts basés sur des ensembles
- Modèle de données (externe, logique, physique)
- LDD et LMD

Atouts d'un SGBD(R)

- Persistance : les données survivent aux programmes
- Taille : accès efficace aux grandes quantités de données : gestion des tampons, index, optimisation des requêtes
- Performances sur le transactionnel (OLTP)
- Standard industrie :
 - SQL2 : indépendance des applications
 - architecture client-serveur
 - optimisation de requêtes
- Sécurité
- Transactions
- Architecture client-serveur

Limite du relationnel

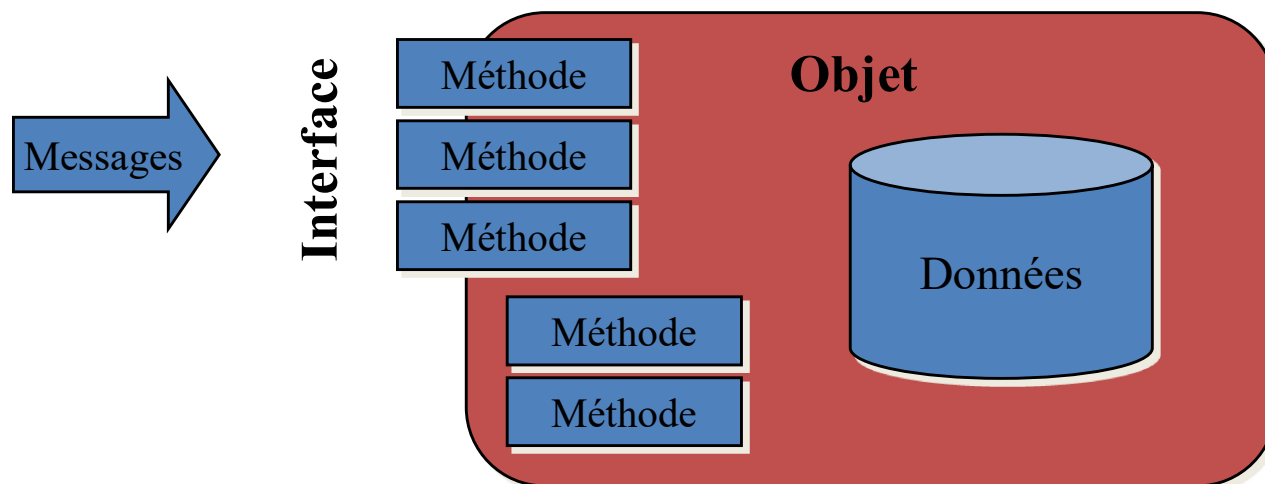
- Modèle plat : pas de données complexes
 - Difficile d'exprimer des choses structurées
 - 1FN
 - √nbre relations
 - Difficile de naviguer dans la structure
 - Jointure
 - -> inadaptés aux application navigationnelles (réseaux)
 - Pas de types nouveaux
 - volumineux ou/et multimédia
 - Non intégration des opérations
- SQL déclaratif, simple
 - → Langage non complet: interfaçage avec un langage de programmation

Définition d'un objet

- Pas de définition universelle de l'objet
 - OMG (Object Management Group), association américaine
 - Chose identifiable
 - Ferber
 - Entité informatique complète identifiable caractérisée pas son état et un ensemble de méthodes

Définition d'un objet (2)

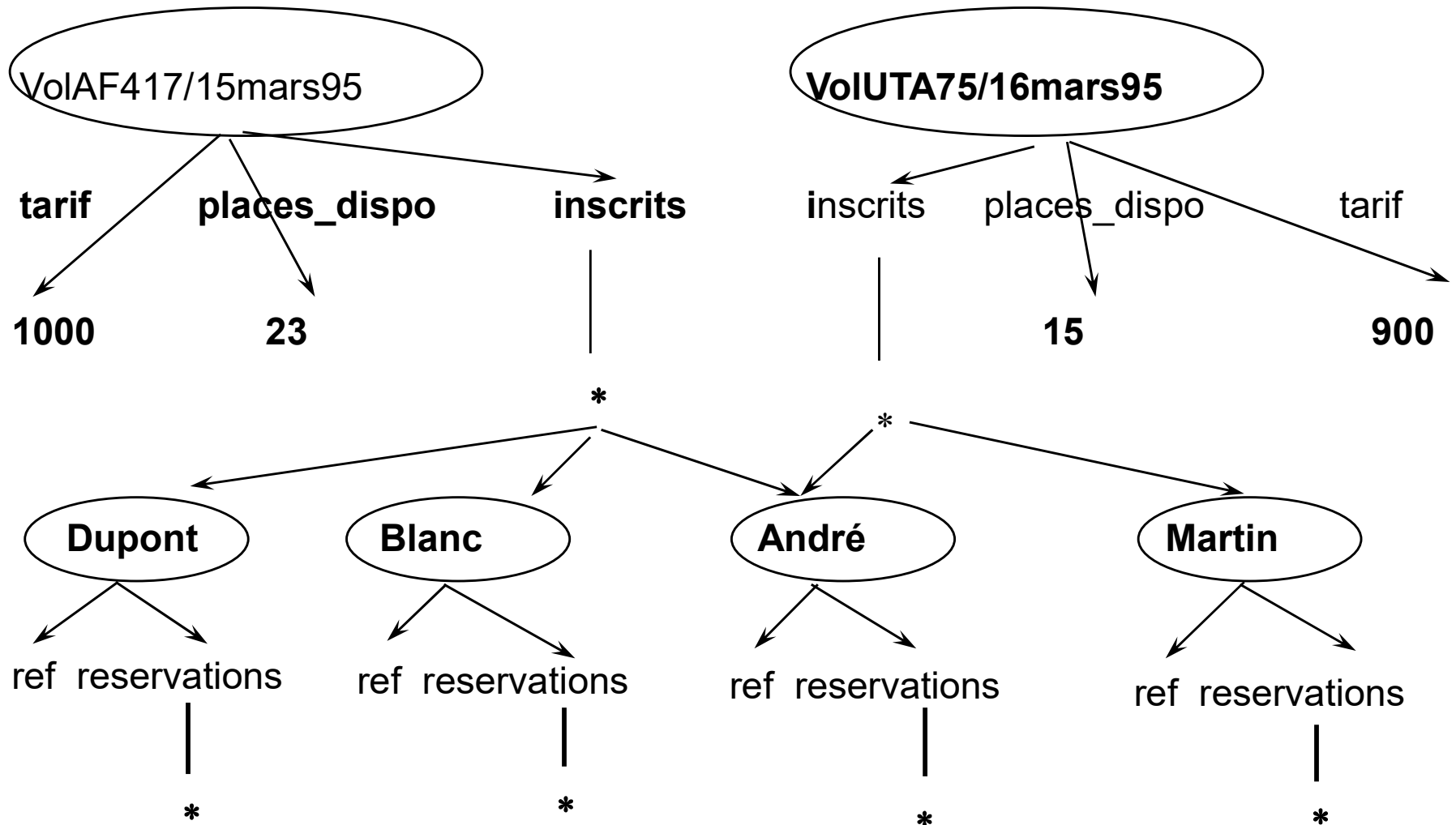
- Associe traitements (méthodes) et données dans une même entité.
- L'interface est le seul moyen d'accéder à l'objet, par l'envoi de messages.



Identité d'objet

- Chaque objet a une identité indépendante de sa valeur.
- On peut mettre à jour la valeur sans changer l'identité.
- Les objets peuvent être partagés.
- Les objets peuvent être cycliques.
- Les identificateurs ne sont pas accessibles aux utilisateurs.

Exemple



Modèle objet pur

- Premier SGBD objet : 1983 (Gemstone)
- Approche : étendre le langage de programmation objet aux fonctions de SGBD
- Persistance
 - orthogonale au type
 - langages : C++, smalltalk, Java/OQL
- Produits
 - O2, ObjectStore, Ontos, Objectivity, Jasmine, Versant
- Niches technologiques
 - réseau, CAO, SIG, Gestion de données Techniques
 - pas de transactionnel lourd

Règles d'or d'un SGBD Objet

Pour être qualifié d'objet, un SGBD doit respecter treize règles [Soutou99]

1. Données persistantes
 - Les données persistantes et non persistantes (temporaires) sont manipulées de la même façon par un programme
2. Grande quantité de données
 - Techniques de regroupement physique, d'indexation, d'optimisation de requêtes et de gestion de cache
3. Fiabilité des données
 - Transactions et privilèges
4. Partages de données
 - multi-utilisateur → mécanismes de verrous
5. Facilité d'interrogation
 - Langage de requêtes.
 - Le résultat d'une requête n'est pas forcément un objet d'une classe existante.
 - Le langage offre en outre la puissance d'un langage de programmation

Règles d'or d'un SGBD Objet

6. Encapsulation

- Les données sont accessibles par des méthodes à différents degrés (visibilité)

7. Objets composites

- Les structures de données à définir et à manipuler peuvent être complexes

8. Identificateur d'objet

- Accès aux objets directement ou par des liens inter-objets via les OID
- Objets partagés

9. Classes, types et méthodes

- Types abstraits de données
- Concepts de classes et de méthodes de l'approche objet

10. Héritage

Règles d'or d'un SGBD Objet

11. Surcharge et liaison dynamique

- Les méthodes peuvent être surchargées. La liaison dynamique est la capacité d'établir la correspondance entre le nom d'une méthode et son implantation lors de l'exécution et non lors de la compilation

12. Langage de programmation complet

- Le système dispose d'un langage de programmation qui ne nécessite pas d'opérateur externe pour écrire une application

13. Extensibilité

- Le système autorise l'ajout dynamique de nouveaux types abstraits de donnée, de nouvelles classes, de nouvelles méthodes, etc.

Atouts de la technologie objet dans les bases de données

- Origine dans les langage de programmation objet
- Objectif principal de l'approche objet : augmenter le niveau d'abstraction.
- Objet dans les SGBD : combiner les avantages
 - fichiers (simplicité et rapidité d'accès),
 - bases hiérarchiques ou réseaux (navigation entre objets)
 - bases relationnelles (langage de requêtes)
- Réduction des différences
 - entre le langage de programmation et la BD,
 - entre le monde à modéliser et le modèle de données d'autre part

Limites de la technologie objet dans les bases de données

- Manque de théorie dans la conception d'un schéma.
Modèle de données :
 - complexe
 - manque de maturité
- Tenue de charge des moteurs lorsque le nombre d'utilisateurs concurrents augmente d'une manière importante

SGBD objet-relationnel

- Un SGBD objet-relationnel doit selon Stonebraker prendre en compte les quatre mécanismes suivants
 - L'extension des types de données
 - Les objets complexes (en terme de structures de données)
 - L'héritage
 - Les systèmes de règles

Propriétés RICE

- Réutilisation
 - finalité du paradigme objet
 - héritage, généralité, composition, polymorphisme
- Identité
 - identifier un objet de manière unique
- Complexité
 - définition d 'objets complexes et/ou fortement structurés
- Encapsulation
 - boîte noire avec des méthodes de manipulation

Le modèle de données

- Le modèle de données objet-relationnel est une extension du modèle de données relationnel
- Une des principales extensions consiste à manipuler des structures de données complexes incluant
 - Des pointeurs (facilitent la navigation)
 - Des tables imbriquées (permettent de s'affranchir de la règle de la première forme normale)
- Les objets sont stockés dans des tables objets-relationnelles qui présentent des caractéristiques semblables au modèle de données NF2 (Non First Normal Form)

**Une table au modèle NF2 peut contenir un attribut
qui est composé d'une liste de valeurs**

Oracle8 et l'objet-relationnel

- Oracle 8 est le premier serveur objet-relationnel d'Oracle
- Un des objectifs d'Oracle est de disposer de produits permettant le développement du commerce électronique basé sur un mécanisme transactionnel
- L'architecture d'Oracle est un montage trois tiers
 - Le client (navigateur Internet)
 - Le serveur applicatif (exécutable)
 - Le serveur de base de données
- Oracle ne met pas encore en oeuvre le mécanisme d'héritage dans la version 8.0.x

La programmation OO avec Oracle

- Dans PL/SQL la programmation orientée-objet est basée sur des types objets (object types).
- Ces types objets fournissent
 - 1) des modèles abstraits pour des objets du monde réel
 - 2) l'encapsulation ("boîte noire").
- Pour utiliser un type objet dans un programme, il suffit de connaître ce qu'il fait et non comment il le fait.

Le Type Objet

- Un type objet est un type de données composé défini par l'utilisateur, qui encapsule une structure de données ainsi que les fonctions et procédures nécessaires à la manipulation des données.
- Les variables qui forment la structure de données sont appelées des attributs et les fonctions/procédures qui caractérisent le comportement du type objet sont les méthodes.
- Un objet a donc des attributs et un comportement.

La définition d'un type objet

```
CREATE [OR REPLACE] TYPE type_name AS OBJECT  
(  att_name datatype [, att_name datatype] ...  
  [ MEMBER {procedure_spec | function_spec}  
  [, MEMBER {procedure_spec | function_spec}] ] ... );
```

```
[CREATE [OR REPLACE] TYPE BODY type_name AS  
  MEMBER {procedure_spec | function_spec};  
  [, MEMBER {procedure_spec | function_spec}; ] ...  
END ; ]
```

Exemple(1)

Creation d'objet:

CREATE TYPE Nbre_rationnel **AS OBJECT**

```
(  Numerateur      INTEGER,
    Denominateur    INTEGER,
    MEMBER FUNCTION plus (x Nbre_rationnel) RETURN Nbre_rationnel,
    MEMBER FUNCTION moins (x Nbre_rationnel) RETURN Nbre_rationnel,
    MEMBER FUNCTION fois (x Nbre_rationnel) RETURN Nbre_rationnel );
```

CREATE TYPE BODY Nbre_rationnel **AS**

```
    MEMBER FUNCTION plus (x Nbre_rationnel) RETURN Nbre_rationnel IS
    r Nbre_rationnel;
```

BEGIN

```
    -- retourne la somme de SELF et x
```

```
    -- appel du CONSTRUCTOR
```

```
    r:= Nbre_rationnel (numérateur * x.denominateur + x.numérateur * denominateur, denominateur * x.
        denominateur );
```

```
    RETURN r;
```

END plus;

...

END;

Exemple(2)

Accès aux attributs d'objet:

DECLARE

emp employee_typ; -- emp est automatiquement null

BEGIN

-- appeler le constructeur pour employee_typ

emp := employee_typ(315, 'Francis', 'Logan', 'FLOGAN', '555.777.2222', '01-MAY-04', 'SA_MAN',
11000, .15, 101, 110, address_typ('376 Mission', 'San Francisco', 'CA', '94222'));

DBMS_OUTPUT.PUT_LINE(emp.first_name || ' ' || emp.last_name); -- display details
emp.display_address(); -- méthode appelée pour afficher les attributs d'un objet

END;

Exemple(3)

Insertion de lignes dans une table d'objets:

DECLARE

emp employee_typ;

BEGIN

INSERT INTO employee_tab VALUES (employee_typ(310, 'Evers', 'Boston', 'EBOSTON',
 '555.111.2222', '01-AUG-04', 'SA_REP', 9000, .15, 101, 110, address_typ('123 Main', 'San
 Francisco', 'CA', '94111')));

INSERT INTO employee_tab VALUES (employee_typ(320, 'Martha', 'Dunn', 'MDUNN',
 '555.111.3333', '30-SEP-04', 'AC_MGR', 12500, 0, 101, 110, address_typ('123 Broadway',
 'Redwood City', 'CA', '94065')));

END;

Exemple(4)

Accès aux méthodes d'objet:

DECLARE

emp employee_typ;

BEGIN

SELECT VALUE(e) INTO emp FROM employee_tab e WHERE e.employee_id = 310;

emp.display_address(); -- méthode appelée pour afficher les attributs d'un objet

END;

Exemple(5)

Mise à jour et suppression de lignes dans une table d'objets:

DECLARE

emp employee_typ;

BEGIN

INSERT INTO employee_tab VALUES (employee_typ(370, 'Robert', 'Myers', 'RMYERS',
 '555.111.2277', '07-NOV-04', 'SA_REP', 8800, .12, 101, 110, address_typ('540 Fillmore', 'San
 Francisco', 'CA', '94011')));

UPDATE employee_tab e SET e.address.street = '1040 California' WHERE e.employee_id = 370;

DELETE FROM employee_tab e WHERE e.employee_id = 310;

END;

Les objets

Une fois un type objet défini et inséré au schéma, il peut être utilisé pour déclarer des objets dans n'importe quel bloc PL/SQL, sous-programme, ou package.

Relationnel vs. objet: le modèle relationnel

POUR

- Le modèle relationnel est adapté à un grand nombre d'applications (de gestion par exemple), qui utilisent des types simples de données.
- Le modèle relationnel est basé sur une théorie "scientifique" et a été mis à l'épreuve ces 15 dernières années:
 - travaux de Codd;
 - ...
- Le modèle relationnel est facile à comprendre.
- ...

CONTRE

- Le modèle relationnel ne permet pas de représenter certaines entités complexes du monde réel.
- Les jointures peuvent être des opérations relativement lourdes.
- ...

Relationnel vs. objet: le modèle objet

POUR

- Les objets permettent de représenter [plus] naturellement des entités du monde réel:
 - objets complexes.
- Les objets offrent une plus grande souplesse pour gérer les structures de données complexes (multimedia, CAO, AGL, etc.)

CONTRE

- Les SGBDO posent un certain nombre de problèmes:
 - gestion des objets en mémoire et dans la base;
 - performance d'accès aux objets;
 - concurrence d'accès aux objets.
- ...